

МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Балтийский государственный технический университет «ВОЕНМЕХ» им.
Д.Ф. Устинова» (БГТУ «ВОЕНМЕХ» им. Д.Ф. Устинова)

РАСЧЁТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА о курсовой работе

Тема Применение параллельных вычислений при работе с базой данных
списка адресов почты

Обучающегося группы О717Б
группа

Праудин Д.С.
Фамилия и инициалы

**Направление подготовки /
специальность** 09.03.04
индекс

Программная инженерия
полное наименование направления подготовки / специальности

**Направленность
образовательной программы**

Разработка программно-информационных систем
профиль / специализация / магистерская программа

Дисциплина (модуль)

Основы параллельных вычислений

Руководитель: _____
подпись

ст. преподаватель
ученая степень, ученое звание

Зими́на Д.В.
Фамилия ИО

Оценка:

« » _____ 20 24 г.

Обучающийся: _____
подпись

Праудин Д.С.
Фамилия ИО

« » _____ 20 24 г.

САНКТ-ПЕТЕРБУРГ
2024 г.

РЕФЕРАТ

Пояснительная записка 26с., 21 рис., 5 источн., 1 прил..

БАЗА ДАННЫХ, ПАРАЛЛЕЛЬНЫЕ ВЫЧИСЛЕНИЯ, ЗАПРОСЫ, ПОТОКИ, ЗАДАНИЯ

Целью курсовой работы является разработка клиентского приложения по работе с базой данных с применением параллельных вычислений. Согласно выбранному варианту, тема работы – база данных журнала сервера.

Объектом исследования является возможность эффективного применения параллельных вычислений при работе с базой данных.

Предметом исследования являются средства СУБД SQLite по созданию базы данных, средства языка программирования C++ по взаимодействию с SQLite и реализации параллельных вычислений.

В ходе работы было произведено создание базы данных, выявлены основные проблемы, которые нужно учитывать при разработке клиентского приложения по работе с базой данных, разобраны основные принципы параллельных вычислений, разработано клиентское приложение по работе с базой данных с использованием параллельных вычислений, проведено тестирование приложения.

В результате выполнения работы было создано готовое клиентское приложение по работе с базой данных журнала сервера с применением параллельных вычислений при обработке информации из базы данных.

СОДЕРЖАНИЕ

РЕФЕРАТ	2
ВВЕДЕНИЕ	4
1 Описание базы данных	5
1.1 Создание базы данных	5
1.2 Работа с базой данных в IDE	8
2 Анализ проблем работы с базой данных.....	10
2.1 Основные проблемы, возникающие при работе с базой данных.....	10
2.2 Необходимость внедрения параллельных вычислений для оптимизации работы с базой данных.....	10
3 Применение параллельных вычислений.....	12
3.1 Описание принципов работы параллельных вычислений	12
3.2 Применение параллельных вычислений	14
4 Реализация и тестирование	16
4.1 Реализация клиентского приложения.....	16
4.2 Тестирование работоспособности приложения.....	21
5 Результаты и выводы	23
ЗАКЛЮЧЕНИЕ	24
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	25

ВВЕДЕНИЕ

Целью данного проекта является исследование и внедрение параллельных вычислений в клиентское приложение для работы с базой данных списка адресов почты. В работе рассматриваются структура базы списка адресов почты, также анализируются проблемы, возникающие при работе с базой данных, и обосновывается необходимость применения параллельных вычислений для их решения, также будет уделено внимание принципам работы параллельных вычислений и их применению в клиентском приложении.

Проект включает в себя этапы проектирования и реализации клиентского приложения, а также тестирование его работоспособности. В заключительной части проекта приведен анализ полученных результатов и сделаны выводы о целесообразности использования параллельных вычислений в работе с базой данных списка адресов почты.

1 Описание базы данных

В данном разделе будет описано создание базы данных. Также будут рассмотрены возможности работы с ней непосредственно из IDE.

1.1 Создание базы данных

Разработанная база данных списка адресов почты представляет из себя хранилище информации о запросах к серверу за определенный промежуток времени. В качестве СУБД была выбрана SQLite. База данных создается непосредственно при запуске программы. Созданную базу данных example можно просмотреть в приложении DB Browser (SQLite), представленном на рисунке 1.

	id	name	email
	Фи...	Фильтр	Фильтр
1	1	Zom15dZqvF98yCrdSEvq	xYB@IGRrho2uiacjgxrI.com
2	2	c3oqFhgJzLDJ9sw0	TZTKdIAZOT6hRf@2yeJETIK4ZWxbLcS4XEe.com
3	3	95vbi7t2v	37Lv8my3Croy8zyH@7rMLxM.com
4	4	tIm3GCigRHZpbS	b@NU3Ye.com
5	5	RkOXMI	VwI6OG@6497ycidmQmVWwKLBs.com
6	6	oQYLQolwU9PLKCbUoTP	wLRchaIcuPETLWc@aBri5E1ZYIEDvkD.com
7	7	1fGcQymAn46	scsRWIdda90MFa@LW36ayI58Z3R3jU.com

Рисунок 1 – Созданная база данных

В базе данных example, содержимое которой представлено на рисунке 2, была создана таблица users.

▼	Таблицы (1)	
▼	users	CREATE TABLE users (id INTEGER PRIMARY KEY,
	id	INTEGER "id" INTEGER
	name	TEXT "name" TEXT NOT NULL
	email	TEXT "email" TEXT NOT NULL
	Индексы (0)	
	Представления (0)	
	Триггеры (0)	

Рисунок 2 – Содержимое базы данных

Кроме таблицы на рисунке можно заметить поля «id», «name» и «email». В СУБД SQLite также как и в других БД полям причисляется их тип. Тип INTEGER один из самых простых примитивных типов данных в информатике. Служит для представления целых чисел, в нашем случае, id элементов БД. Тип TEXT это тип данных в SQL, который используется для хранения больших объемов текста.

Были созданы следующие поля таблицы:

- id, автоматически заполняемое поле, обозначающее номер записи в таблице;
- name, автоматически заполняемое программой поле из случайных символов;
- email, автоматически заполняемое программой поле из случайных символов с добавлением обязательного текста, такого как «@» и «.com».

Ключевым полем данной таблицы будет id, так как именно по нему можно определить, какой элемент БД нам нужен.

Таблица была создана с помощью программного кода, приведенного на рисунке 3.

```
sqlite3* db;
int rc = sqlite3_open("example.db", &db);

if (rc != SQLITE_OK) {
    std::cerr << "Error opening the database: " << sqlite3_errmsg(db) << std::endl;
    return 1;
}

std::cout << "The database has been opened successfully." << std::endl;

const char* create_table_query = "CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY, name
if (sqlite3_exec(db, create_table_query, NULL, NULL, NULL) != SQLITE_OK) {
    std::cerr << "Ошибка при создании таблицы: " << sqlite3_errmsg(db) << std::endl;
    sqlite3_close(db);
    return 1;
}

std::cout << "Error creating the table." << std::endl;
```

Рисунок 3 –Программный код для создания таблицы

Описанные ранее поля таблицы users продемонстрированы на рисунке 4.

	id	name	email
	Фи...	Фильтр	Фильтр
34	34	LwSXMjwQha	hfDy0jR7w67v1wC@BBjzqSYhPpuLV.com
35	35	F7Q	qGRRhSE6@RqgdZv4ii2XMn.com
36	36	vEhKY	b0Zf7sTVAfIkT9y7aemf@uwHqwZvWojl0j7507.com
37	37	2OuY6B8IX	LxtznyKsjErzE6iKkP@WS7aT.com
38	38	MkgOHA42e5bFdWo	MLiLSFa@eC.com
39	39	kbVgRbZdHU	VG@Ct.com

Рисунок 4 – Поля созданной таблицы

Таблица была заполнена случайными данными с помощью программного кода. Было добавлено 1000 записей. Программный код, использованный для заполнения таблицы приведен на рисунке 5.

```
// Функция для вставки данных в базу данных
void insert_data(sqlite3* db, int num_entries, int thread_id) {
    std::string query = "INSERT INTO users (name, email) VALUES (?, ?)";
    sqlite3_stmt* stmt;

    if (sqlite3_prepare_v2(db, query.c_str(), -1, &stmt, NULL) != SQLITE_OK) {
        std::cerr << "Thread " << thread_id << ": Error in preparing the request: " << sqlite3_errmsg(db) << std::endl;
        return;
    }

    for (int i = 0; i < num_entries; ++i) {
        std::string name = generate_random_string();
        std::string email = generate_random_email();

        sqlite3_bind_text(stmt, 1, name.c_str(), -1, SQLITE_TRANSIENT);
        sqlite3_bind_text(stmt, 2, email.c_str(), -1, SQLITE_TRANSIENT);

        if (sqlite3_step(stmt) != SQLITE_DONE) {
            std::cerr << "Thread " << thread_id << ": Error in preparing the request: " << sqlite3_errmsg(db) << std::endl;
        }
        else {
            std::cout << "Thread " << thread_id << ": Inserted (" << name << ", " << email << ")" << std::endl;
        }

        sqlite3_reset(stmt);
    }

    sqlite3_finalize(stmt);
}
```

Рисунок 5 – Добавление записей в таблицу

1.2 Работа с базой данных в IDE

В IDE для разработки на языке C++ - Visual Studio 2022 доступна работа с базами данных на базе расширения SQLite. Соответствующее расширение представлено на рисунке 6.

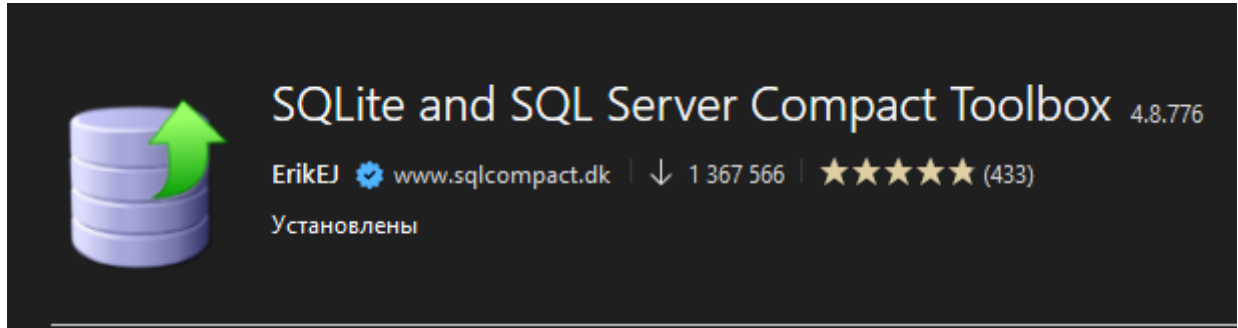


Рисунок 6 – Расширение для Visual Studio 2022

К сожалению, в Visual Studio 2022 нельзя просто так просмотреть БД, но можно воспользоваться сторонними IDE или программами, например такая как DB Browser (SQLite). Это продемонстрировано на рисунке 7.

344	344	m4PsWeP0F	odbKnwxTBxE4@6QdSX.com
345	345	O	X7JV@AqgnJ.com
346	346	FeOAnXrasbfKiII	Bkwejj@Hbujvk6F1JTRc6IKc.com
347	347	V0YtdOpN8glga8env2d1	Xhq1PpN8r2DfgFHwZi@rVKX4Cs9G.com
348	348	M7kfRn2nMt	aeEHY1AMW@MA7jwgMUiKa1Iy81stX.com
349	349	Td	qoJcI@5vxw2.com
350	350	GLM7	dH6y4LmWLV2gT03L@i0v1GMVJX.com
351	351	j9L	p@pqFy0zs.com
352	352	jb4nLeVSdt3sI	Qlhm@VShjvFiqZq0D.com
353	353	FcXppYYi	nGOPlvC2TXUKzbj58unA@R.com
354	354	m5z4wCXjkX2UtGj	4buUwDpLX0qhnbsjr@nc1wsP.com

Рисунок 7 – Содержимое таблицы в DB Browser (SQLite)

Для подключения базы данных в проекте был создан специальный участок кода, изображенный на рисунке 8.


```
int main() {  
    sqlite3* db;  
    int rc = sqlite3_open("example.db", &db);  
  
    if (rc != SQLITE_OK) {  
        std::cerr << "Error opening the database: " << sqlite3_errmsg(db) << std::endl;  
        return 1;  
    }  
    std::cout << "The database has been opened successfully." << std::endl;  
}
```

Рисунок 8 – Создание класса для подключения базы данных

2 Анализ проблем работы с базой данных

В данном разделе рассмотрены основные проблемы, возникающие при работе с базой данных, также будет рассмотрена необходимость внедрения параллельных вычислений для оптимизации работы с базой данных.

2.1 Основные проблемы, возникающие при работе с базой данных

Основные проблемы при работе с базой данных, которые необходимо учитывать при разработке клиентского приложения по работе с базой данных:

- проблемы производительности такие, как медленные запросы и операции, особенно при обработке большого объема данных, неэффективное использование индексов и неоптимизированные запросы;
- проблемы надежности и отказоустойчивости такие, как потеря данных из-за аппаратных сбоев или ошибок программного обеспечения и нарушение целостности данных из-за неправильного использования транзакций или нарушения ограничений;
- проблемы безопасности и конфиденциальности – атаки, такие как SQL-инъекции, которые могут привести к компрометации данных. Необходимость использовать параметризованные запросы.

2.2 Необходимость внедрения параллельных вычислений для оптимизации работы с базой данных

В коде используется несколько потоков для параллельной вставки данных в базу данных SQLite. Это может быть полезно, особенно если вставляемые данные большие или если вставка занимает много времени. Однако, следует учитывать, что SQLite имеет свои собственные механизмы блокировки, которые могут снизить эффективность параллельной вставки данных, особенно при одновременной записи в одну и ту же таблицу. В таких случаях может быть полезно провести тестирование и оценить реальное ускорение при использовании параллельной вставки.

В данном коде нет явного использования параллельных вычислений для поиска данных в базе данных. Однако, в зависимости от сложности запросов и объема данных, параллельный поиск может значительно ускорить

выполнение программы. SQLite поддерживает многопоточный доступ к базе данных, но стоит помнить о том, что одновременный доступ к базе данных из нескольких потоков может потребовать использования механизмов синхронизации для предотвращения конфликтов.

Общий подход к оптимизации работы с базой данных SQLite включает в себя анализ производительности, оценку узких мест и выбор наиболее подходящих стратегий оптимизации, включая параллельные вычисления.

3 Применение параллельных вычислений

В данной разделе описаны принципы работы параллельных вычислений и применение параллельных вычислений в клиентском приложении для работы с базой данных журнала сервера.

3.1 Описание принципов работы параллельных вычислений

Принцип работы параллельных вычислений в данном коде основан на использовании многопоточности для выполнения нескольких задач одновременно. В данном случае, используются потоки для параллельной вставки данных в базу данных SQLite.

Основные принципы работы параллельных вычислений в этом коде:

1) разделение задач: Задача вставки данных разделяется на несколько подзадач, каждая из которых будет выполняться в отдельном потоке. В данном случае, количество записей для вставки делится между несколькими потоками, чтобы ускорить процесс вставки;

2) параллельное выполнение: Потоки выполняются параллельно на многоядерном процессоре (если таковой имеется). Каждый поток работает независимо друг от друга, что позволяет использовать вычислительные ресурсы системы более эффективно;

3) синхронизация: Важным аспектом параллельных вычислений является правильная синхронизация между потоками. В данном случае, каждый поток имеет доступ к базе данных SQLite, и чтобы избежать гонок данных и других проблем, необходимо правильно управлять доступом к ресурсам базы данных. SQLite имеет встроенную поддержку многопоточности, но нужно быть внимательным при использовании одного и того же соединения с базой данных из разных потоков;

4) объединение результатов: После завершения выполнения всех потоков, необходимо объединить результаты и выполнить необходимые действия. В данном коде после завершения параллельной вставки данных, программа продолжает выполнение и предоставляет пользователю возможность выполнить различные запросы к базе данных.

Методы реализации параллельных вычислений:

- 1) **многопоточность:** многопоточность предполагает выполнение нескольких потоков (threads) внутри одного процесса, каждый поток может выполнять свою задачу, при этом все потоки могут работать параллельно на разных ядрах процессора;
- 2) **многопроцессность:** многопроцессность предполагает выполнение нескольких процессов, каждый из которых имеет свое собственное адресное пространство, позволяет изолировать процессы друг от друга, что повышает стабильность и безопасность выполнения;
- 3) **распределенные вычисления:** в распределенных вычислениях задачи выполняются на нескольких машинах, объединенных в сеть, позволяет масштабировать вычисления и использовать ресурсы нескольких машин для выполнения сложных задач;
- 4) **графические процессоры:** графические процессоры могут использоваться для параллельных вычислений благодаря большому числу ядер, способных выполнять простые операции одновременно, особенно полезно для задач, требующих больших объемов вычислений, таких как обработка изображений или машинное обучение.

Преимущества параллельных вычислений:

- **увеличение скорости выполнения:** параллельное выполнение задач позволяет значительно сократить время обработки данных по сравнению с последовательным выполнением;
- **эффективное использование ресурсов:** параллельные вычисления позволяют максимально использовать доступные вычислительные ресурсы, такие как многоядерные процессоры и распределенные системы;
- **масштабируемость:** возможность распределения задач между несколькими машинами позволяет легко масштабировать вычисления в зависимости от требований задачи.

Параллельные вычисления являются мощным инструментом для повышения производительности и эффективности программного

обеспечения. Они позволяют выполнять сложные задачи быстрее и более эффективно, за счет одновременного выполнения нескольких подзадач. Правильное применение принципов параллельных вычислений, таких как декомпозиция задач, параллельное исполнение, синхронизация и управление задачами, позволяет значительно улучшить производительность и масштабируемость вычислительных процессов.

3.2 Применение параллельных вычислений

Для улучшения производительности работы приложения можно применить параллельные вычисления. Одним из способов это сделать является использование многопоточности для одновременного выполнения большого количества однотипных операций.

Для реализации параллельности было использовано многопоточное создание и выполнение задач: В цикле `for` создаются несколько потоков с использованием стандартной библиотеки потоков C++, которые вызывают функцию `«insert_data»`. Каждый поток выполняет свои вычисления параллельно другим потокам.

```
for (int i = 0; i < num_threads; ++i) {  
    threads.emplace_back(insert_data, db, num_entries / num_threads, i);  
}
```

Распределение работы между потоками: Количество записей для вставки делится между созданными потоками таким образом, чтобы каждый поток обработал примерно одинаковое количество записей. Это позволяет равномерно распределить нагрузку между потоками.

```
threads.emplace_back(insert_data, db, num_entries / num_threads, i);
```

Синхронизация потоков: После создания всех потоков основной поток программы ожидает завершения выполнения каждого потока с помощью вызова функции `join()`. Это гарантирует, что основной поток не завершится до завершения всех созданных потоков.

```
for (auto& thread : threads) {  
    thread.join();  
}
```

Таким образом, эти элементы обеспечивают параллельное выполнение задачи вставки данных в базу данных SQLite с использованием нескольких потоков, что позволяет ускорить выполнение программы и более эффективно использовать ресурсы вычислительной системы.

4 Реализация и тестирование

В данной главе будет рассматриваться проектирование и реализация клиентского приложения с использованием параллельных вычислений, также будет проведено тестирование работоспособности приложения.

4.1 Реализация клиентского приложения

Приложение было разработано на языке программирования C++ с использованием СУБД SQLite.

Приложение состоит из следующих функций:

- `generate_random_string` – функция для генерации случайных строк;
- `generate_random_email` – функция для генерации случайного email;
- `clear_table` – функция для очистки таблицы;
- `insert_data` – функция для вставки данных в базу данных;
- `main` – функция, из которой запускается программа;
- `execute_select_query`, функция для выполнения запроса SELECT по критериям `name` или `email`.

Созданные функции в структуре приложения продемонстрированы на рисунке 9.

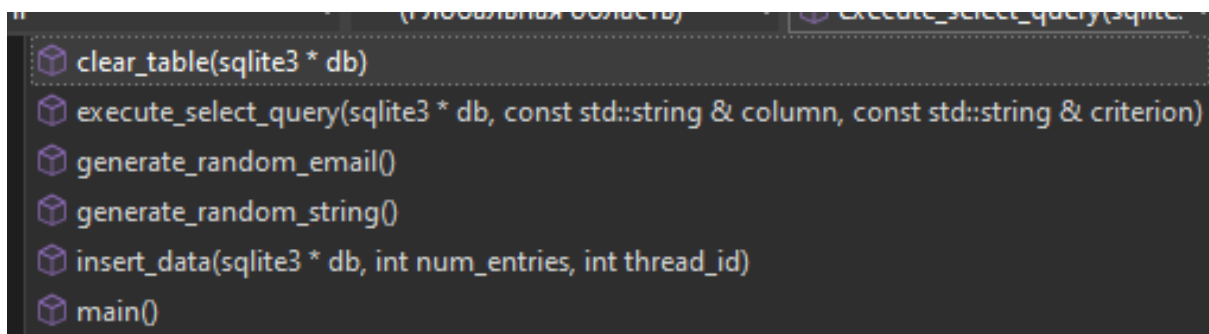


Рисунок 9 – Структура проекта

Функция `generate_random_string`, генерирует случайные строки для полей «name» и «email», в ней определяется массив символов «alphanumeric», который содержит все символы, из которых будет состоять случайная строка, с созданием случайной длины строки генератором случайных чисел, показана на рисунке 10.


```

// Функция для генерации случайных строк
std::string generate_random_string() {
    static const char alphanum[] =
        "0123456789"
        "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
        "abcdefghijklmnopqrstuvwxyz";

    std::string str;
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> dis(1, 20);

    int length = dis(gen);
    for (int i = 0; i < length; ++i) {
        str.push_back(alphanum[gen() % (sizeof(alphanum) - 1)]);
    }

    return str;
}

```

Рисунок 10 – Функция для генерации случайных строк

Функция `generate_random_email` используется для создания случайной части адреса электронной почты (до символа «@») функция вызывает другую функцию `generate_random_string`. Эта функция генерирует случайную строку определенной длины, состоящую из цифр и букв алфавита. После генерации случайной строки функция добавляет символ «@» и затем вызывает `generate_random_string` еще раз, чтобы создать случайную часть второй части адреса (после символа «@») вместе с «.com». Функция для генерации случайного email представлена на рисунке 11.

```

// Функция для генерации случайного email
std::string generate_random_email() {
    return generate_random_string() + "@" + generate_random_string() + ".com";
}

```

Рисунок 11 – Функция для генерации случайного email

Функция `clear_table`, отвечает за очистку содержимого таблицы в базе данных. Она выполняет SQL-запрос для удаления всех записей из таблицы пользователей («users»), формируя SQL-запрос на удаление всех записей из таблицы пользователей: «DELETE FROM users». Функция `clear_table` представлена на рисунке 12.

```

// Функция для очистки таблицы
void clear_table(sqlite3* db) {
    const char* query = "DELETE FROM users";
    char* errmsg;
    int rc = sqlite3_exec(db, query, NULL, NULL, &errmsg);
    if (rc != SQLITE_OK) {
        std::cerr << "Error clearing the table: " << errmsg << std::endl;
        sqlite3_free(errmsg);
    }
    else {
        std::cout << "The table was cleared successfully." << std::endl;
    }
}

```

Рисунок 12 – Функция для очистки таблицы

В функции main запускается вся программа, также в ней создано меню для выбора объекта анализа. Пользователю предлагается следующий функционал:

- получить отчет по элементам, которые начинаются с определённого символа («name»). Программа посчитает, сколько было таких запросов, и выдаст результат в консоль;
- получить отчет по элементам, которые начинаются с определённого символа («email»). Программа посчитает количество отправленных запросов из адреса почты и выдаст результат в консоль.

Главное меню представлено на рисунке 13.

```

// Обработка пользовательского ввода и выполнение соответствующих запросов
while (true) {
    std::cout << "Select an action:" << std::endl;
    std::cout << "(1) Find elements in <name> that start with a specific character" << std::endl;
    std::cout << "(2) Find items in <email> that start with a specific character" << std::endl;
    std::cout << "(0) Exit" << std::endl;

    int choice;
    std::cin >> choice;

    if (choice == 0) {
        break;
    } else if (choice == 1) {
        std::string symbol;
        std::cout << "Enter a character: ";
        std::cin >> symbol;
        execute_select_query(db, "name", symbol);
    } else if (choice == 2) {
        std::string symbol;
        std::cout << "Enter a character: ";
        std::cin >> symbol;
        execute_select_query(db, "email", symbol);
    } else {
        std::cout << "Incorrect choice." << std::endl;
    }
}

```

Рисунок 13 – Главное меню

Функция `insert_data` отвечает за вставку данных в базу данных. В контексте кода она используется для многопоточной вставки случайно сгенерированных записей в таблицу базы данных SQLite.

Формируется SQL-запрос на вставку данных в таблицу пользователей. Запрос выглядит следующим образом: «INSERT INTO users (name, email) VALUES (?, ?)». Создается подготовленное SQL-выражение с помощью функции `sqlite3_prepare_v2`.

В цикле для каждой записи генерируются случайные имя и адрес электронной почты с помощью функций `generate_random_string` и `generate_random_email`. Значения имени и адреса электронной почты привязываются к параметрам подготовленного выражения с помощью функции `sqlite3_bind_text`. Выполняется SQL-запрос на вставку данных в базу данных с помощью функции `sqlite3_step`. Если запрос выполняется успешно, выводится сообщение о том, что данные были вставлены. Подготовленное SQL-выражение сбрасывается и освобождается с помощью функции `sqlite3_finalize`.

Функция для вставки данных в базу данных приведена на рисунке 14.

```
// Функция для вставки данных в базу данных
void insert_data(sqlite3* db, int num_entries, int thread_id) {
    std::string query = "INSERT INTO users (name, email) VALUES (?, ?)";
    sqlite3_stmt* stmt;

    if (sqlite3_prepare_v2(db, query.c_str(), -1, &stmt, NULL) != SQLITE_OK) {
        std::cerr << "Thread " << thread_id << ": Error in preparing the request: " << sqlite3_errmsg(db) << std::endl;
        return;
    }

    for (int i = 0; i < num_entries; ++i) {
        std::string name = generate_random_string();
        std::string email = generate_random_email();

        sqlite3_bind_text(stmt, 1, name.c_str(), -1, SQLITE_TRANSIENT);
        sqlite3_bind_text(stmt, 2, email.c_str(), -1, SQLITE_TRANSIENT);

        if (sqlite3_step(stmt) != SQLITE_DONE) {
            std::cerr << "Thread " << thread_id << ": Error in preparing the request: " << sqlite3_errmsg(db) << std::endl;
        }
        else {
            std::cout << "Thread " << thread_id << ": Inserted (" << name << ", " << email << ")" << std::endl;
        }

        sqlite3_reset(stmt);
    }

    sqlite3_finalize(stmt);
}
```

Рисунок 14 – Функция для вставки данных в базу данных

Функция `execute_select_query` выполняет запрос `SELECT` к базе данных SQLite с определенными критериями поиска и выводит результаты запроса в консоль.

Формируется SQL-запрос на выборку данных из таблицы пользователей в зависимости от переданных критериев поиска. Запрос имеет вид `"SELECT * FROM users WHERE <column> LIKE '<criteria>%',` где `<column>` - это столбец (например, `name` или `email`), по которому выполняется поиск, а `<criteria>` - это критерий поиска (например, символ или строка, с которой должны начинаться значения в выбранном столбце).

Создается подготовленное SQL-выражение с помощью функции `sqlite3_prepare_v2`. Выполняется SQL-запрос на выборку данных с помощью функции `sqlite3_step`. По мере получения результатов запроса из базы данных, каждая строка выводится в консоль, отображая значения всех столбцов для каждой строки. Считается количество найденных элементов. После завершения запроса подготовленное SQL-выражение сбрасывается и освобождается с помощью функции `sqlite3_finalize`.

Функция для выполнения запроса `SELECT` по критериям `name` или `email` на рисунке 15.

```
// Функция для выполнения запроса SELECT по критериям name или email
void execute_select_query(sqlite3* db, const std::string& column, const std::string& criterion) {
    std::string query = "SELECT * FROM users WHERE " + column + " LIKE '" + criterion + "%'";
    sqlite3_stmt* stmt;

    if (sqlite3_prepare_v2(db, query.c_str(), -1, &stmt, NULL) != SQLITE_OK) {
        std::cerr << "Error preparing the SELECT request: " << sqlite3_errmsg(db) << std::endl;
        return;
    }

    int count = 0;

    std::cout << "Results of the SELECT query:" << std::endl;
    while (sqlite3_step(stmt) == SQLITE_ROW) {
        count++;
        std::cout << "ID: " << sqlite3_column_int(stmt, 0) << ", Name: " << sqlite3_column_text(stmt, 1)
            << ", Email: " << sqlite3_column_text(stmt, 2) << std::endl;
    }

    std::cout << "Elements found: " << count << std::endl;

    sqlite3_finalize(stmt);
}
```

Рисунок 15 – Функция для выполнения запроса `SELECT` по критериям `name` или `email`

4.2 Тестирование работоспособности приложения

При запуске программы в консоли отображается вставка данных в БД, а затем предоставляется возможность пользоваться главным меню. После выбора одного из пунктов меню (кроме выхода) не требуется заново запускать программу, т.к. меню зациклено до момента выбора выхода из программы.

Меню и вставка данных продемонстрировано на рисунке 16.

```
Thread 3: Inserted (pno1nQmy13, 10@ccv1d0t.com)
Thread 1: Inserted (LQ6K, Hk5V93HH995@oCywEqh9vFHuB0.com)
Thread 0: Inserted (VdTG9MsBSLLT7Vf5e, BHpAwM@Xdz1N6hjG.com)
Thread 2: Inserted (m8tKPD5973afMLXy, EJfmUeJDKJqua@eBvF1.com)
Thread 3: Inserted (1Pj21qPbSkxqrU2ujilv, bt@V8zwks0b1.com)
Thread 1: Inserted (vtUJgWEYSYQ7SQG, TcidLvdG4dR914vy3MM@gTiQj.com)
Thread 0: Inserted (oKc4C8ewRM267bDZ, ZLJsojB1D6sZjfFA@QZ9QovJivJo5m.com)
Thread 2: Inserted (j50x0AvW4qUHWtPxNP, K6b1pt@dwS6XWo.com)
Thread 1: Inserted (n8gy0ari5F8Yw05, 8ceDK6awJLJV4p1RqRt@Q5QtrOXnsq8jL.com)
Thread 2: Inserted (y25Huo4zf67YxJeQjX, 3nDFjFDck2j@8SiFjGuU.com)
Thread 1: Inserted (ASQ5X5Aay9p8YDAh, TxNRU9NP@1at496Ns.com)
Data insertion is complete.
Select an action:
(1) Find elements in <name> that start with a specific character
(2) Find items in <email> that start with a specific character
(0) Exit
```

Рисунок 16 – Главное меню и вставка данных в консоли

Далее произведем тестирование каждого пункта меню. Результаты тестирований первого пункта продемонстрированы на рисунках 17-18.

```
Select an action:
(1) Find elements in <name> that start with a specific character
(2) Find items in <email> that start with a specific character
(0) Exit
1
Enter a character: s
```

Рисунок 17 – Отчет по статусу ответа

```
ID: 668, Name: Shnbu7yrs06M2DA4s04, Email: mOjHVHOXqQPXp@2D86qiIvIKm5Jg.com
ID: 724, Name: SxYN, Email: J5fPzWFhfZ2wMnfvatBL@49r8ZJtoFlU8Y24BK.com
ID: 726, Name: sa4jX2dtCITm, Email: l2GqVACQXPol00PK9BUN@JBvhL.com
ID: 777, Name: SMlJc30, Email: yEIOz@VudWRPL0Y.com
ID: 793, Name: shtGzvUaToY13, Email: TyEZgL@Q2ewhFmnX9y5oeap0a.com
ID: 796, Name: seHZp68dNwAnH, Email: jM0V@jB4BX3xXYVnR.com
ID: 822, Name: SSA, Email: oyQd1aKClD@pNz8iN.com
ID: 896, Name: SP5JfdLc1fGyaUC2KuY, Email: jJooD0wcS8p@mLMJ6LrpThuch9.com
ID: 905, Name: sTJMq, Email: Rv72Jfk@J1NbTODnHjh3Da.com
ID: 956, Name: s, Email: u@AffKlEIy5E8MKb.com
Elements found: 29
Select an action:
(1) Find elements in <name> that start with a specific character
(2) Find items in <email> that start with a specific character
(0) Exit
```

Рисунок 18 – Число по именам

Результаты тестирований второго пункта продемонстрированы на рисунках 19-20.

```
Select an action:
(1) Find elements in <name> that start with a specific character
(2) Find items in <email> that start with a specific character
(0) Exit
2
Enter a character: GG
```

Рисунок 19 – Отчет по методу запроса

```
Results of the SELECT query:
ID: 547, Name: ETDx2iJK5wmRGfjdS, Email: GG0WL5Am@8.com
Elements found: 1
Select an action:
(1) Find elements in <name> that start with a specific character
(2) Find items in <email> that start with a specific character
(0) Exit
```

Рисунок 20 – Число адресов

Выход из программы продемонстрирован на рисунке 21.

```
Select an action:
(1) Find elements in <name> that start with a specific character
(2) Find items in <email> that start with a specific character
(0) Exit
0
The connection to the database is closed.

C:\Users\user123456789101\source\repos\KRP11\x64\Debug\KRP11.exe (процесс 28332) завершил работу с кодом 0.
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" ->"Параметры" ->"Отладка".
Нажмите любую клавишу, чтобы закрыть это окно:
```

Рисунок 21 – Выход из программы

По результатам тестирования можем сделать вывод о том, что программа корректно реализует весь функционал.

5 Результаты и выводы

Тестирование показало, программа работает корректно.

Использование многопоточности позволяет ускорить вставку большого объема данных в базу данных. Каждый поток отвечает за вставку своей порции данных, что позволяет распараллелить работу и использовать многопоточность для полной загрузки вычислительных ядер процессора.

В коде используется одно подключение к базе данных, которое используется всеми потоками для выполнения запросов. Это может вызвать проблемы с конкурентным доступом к базе данных, такими как блокировки или гонки данных. Однако, SQLite обеспечивает многопоточный доступ, используя механизмы блокировки, чтобы гарантировать целостность данных.

Для оценки эффективности параллельной вставки данных можно провести эксперименты с разными количествами потоков и объемами данных, а также сравнить время выполнения операции вставки данных при использовании многопоточности и при использовании однопоточного подхода.

Для дальнейшего улучшения производительности можно рассмотреть оптимизацию работы с базой данных, такую как использование транзакций для пакетной вставки данных или предварительную компиляцию SQL-запросов для уменьшения накладных расходов на выполнение запросов.

При использовании многопоточности важно тщательно тестировать код и обрабатывать возможные ошибки, такие как непредвиденное поведение базы данных из-за конкурентного доступа или возможные сбои в работе программы из-за некорректного использования потоков.

В целом, использование параллельных вычислений для работы с базой данных может значительно улучшить производительность и эффективность программы, особенно при обработке больших объемов данных. Однако необходимо быть внимательным к потенциальным проблемам конкурентного доступа и тщательно тестировать код для обеспечения корректной и стабильной работы.

ЗАКЛЮЧЕНИЕ

В ходе выполнения данного проекта была проведена комплексная работа по исследованию и внедрению параллельных вычислений в клиентское приложение для работы с базой данных списка адресов почты.

Были подробно рассмотрены принципы работы параллельных вычислений и их применение в контексте клиентского приложения. Реализация параллельных вычислений позволила повысить эффективность выполнения процесса обработки информации из базы данных.

Таким образом, применение параллельных вычислений в работе с базой данных списка адресов почты является обоснованным и целесообразным решением.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. SQLite — Википедия [Электронный ресурс] – <https://ru.wikipedia.org/wiki/SQLite> (дата обращения 01.06.2024).
2. An Introduction To The SQLite C/C++ Interface [Электронный ресурс] – URL: <https://www.sqlite.org/cintro.html/> (дата обращения 01.06.2024).
3. Биллиг В. А. Параллельные вычисления и многопоточное программирование // В. А. Биллиг. – 2016.
4. Каропова Е. Основы многопоточного и параллельного программирования. – Litres, 2022.
5. Использование SQLite в проектах Visual Studio [Электронный ресурс] – URL: <https://learn.microsoft.com/ru-ru/shows/mvp-azure/using-sqlite-in-visual-studio-projects> (дата обращения 02.06.2024).